

Chapter 6

Arithmetic Unit (20)

1) Explain signed and unsigned number representation.
or
Signed and unsigned integer notation.

- Unsigned number representation

→ It is used for non-negative numbers.

→ It only deals with positive values and does not consider negative numbers.

→ Its advantages is it is simple to understand and use for positive values.

→ It is limited to addition and subtraction.

→ Example:

In a 6 bit unsigned representation, 001011.

001011 → it represents 11.

→ The left most bit is most significant bit (MSB) and right most is least significant bit (LSB).

- Signed number representation

→ It is used for both positive and negative numbers.

→ It uses left most bit (usually MSB) to represent the sign: 0 for positive and 1 for negative.

→ The remaining bits represent the magnitude of the number.

→ It supports addition, subtraction and multiplication.

Example: Represent -10 in a 4 bit signed binary using two's complement.

Step 1: Binary representation of +10 i.e

1010

Step 2: Invert all bits,

0101

Step 3: Add 1 in step 2 result

0101

+1

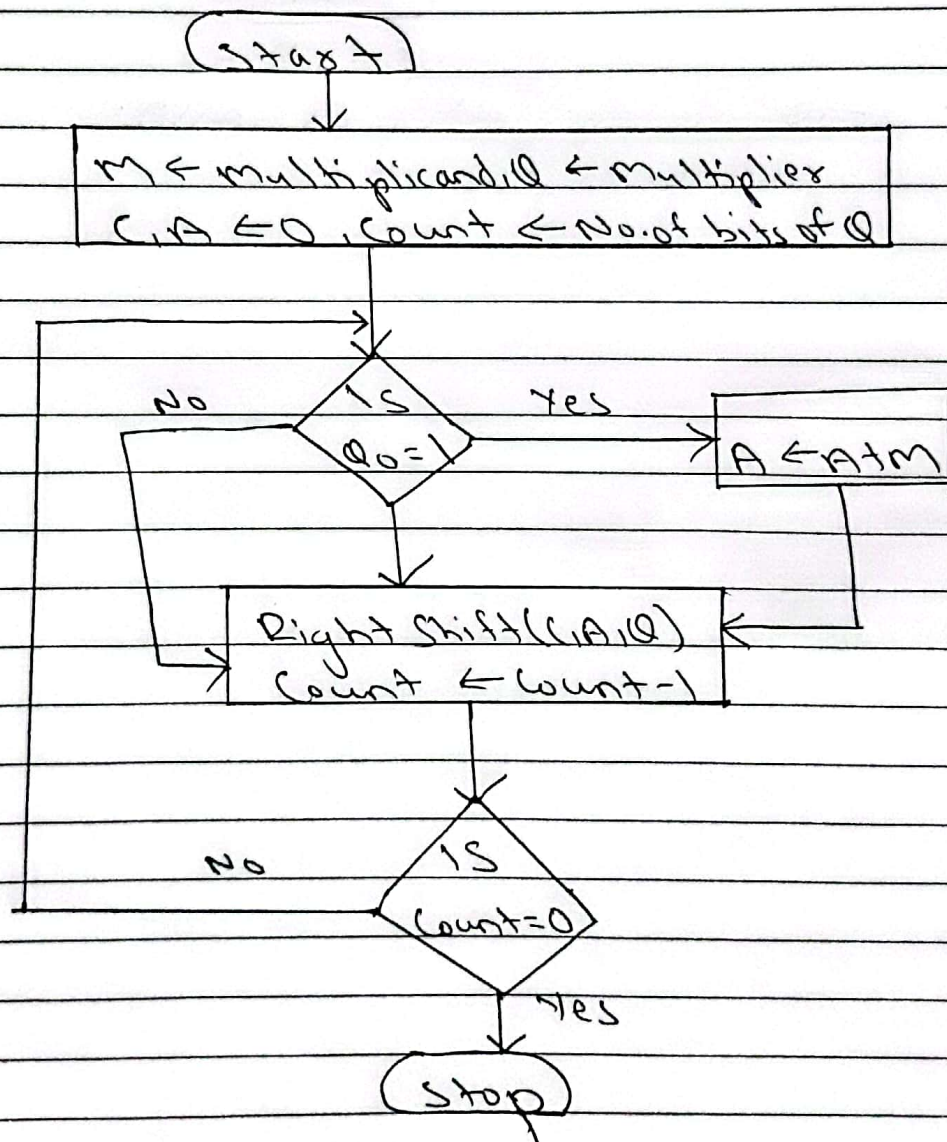
0110

$\therefore$ -10 is represented as 0110.

2) Write the RTL code for shift add multiplication and explain about the algorithm.

→ This algorithm is used for multiplication of unsigned numbers.

→ Multiplication can be performed by repeated addition of multiplicand and shifting.



here, C → carry
A → Accumulator

RTL code

```

1:  $C \leftarrow 0, A \leftarrow 0, \text{count} \leftarrow n$ 
 $\rightarrow$  if ( $Q_0 = 1$ )
    2:  $A \leftarrow A + M$ 
    3:  $\text{shr}(C, A, Q)$ 
    3:  $\text{count} \leftarrow \text{count} - 1$ 
    if ( $\text{count} > 0$ )
        ..... 4: goto 2 if statement of 2
    else
        4: finish

```

3) Trace the RTL code of shift add multiplication algorithm for multiplication of:

- | | |
|----------------------------|------------------------------------|
| a) $x = 0110$ & $y = 0111$ | } 5 Que asked in 5 different sets. |
| b) 4 and 3 | |
| c) 3 and 4 | |
| d) 7 and 9 | |

{ ~~One~~ Question only one question will be asked ~~for~~ for 7 marks }

I will solve only one of them as the process is same for all.

V. Imp

Q. 4) Explain booth algorithm with diagram.
Write the RTL code for Booth's Algorithm.

OR

Write the steps for Booth algorithm.

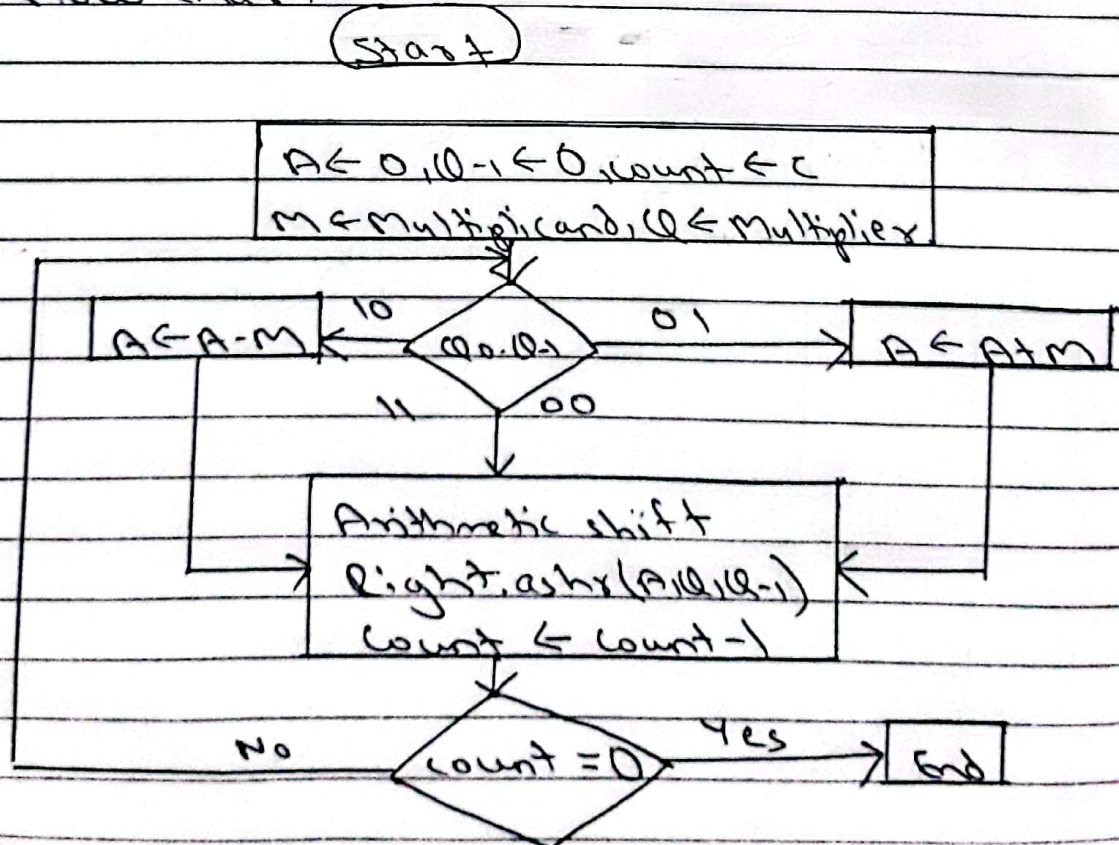
Ans: Booth algorithm is used for multiplication of two binary numbers.

→ It is used to multiply both positive and negative numbers.

→ It helps to reduce the number of operations needed for multiplication.

→ For negative numbers multiplication, the operation is performed by representing the negative number in its 2's complement form.

Flow-chart:



Here, A = Accumulator

→ Here while ~~taking~~ ^{considering} count, first we ~~take~~ consider bit of larger number and add one on it.

i.e. if number is 11 then (1011)

4 bit, $4+1 = 5$

We take 5 bit register.

→ Note: For negative number we take 2's complement

Example: ~~-9~~ ⁻⁹

a. ~~its~~ Binary equivalent of ~~9~~ ⁹ is 1001

b. Invert all bits

1001 will be

0110

c. add 1

0110

+1

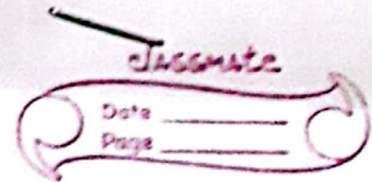
0111

∴ -9 is represented as 0110

Note: if answer should be in positive then the final output is the final result.

if answer should be in negative then ~~the final output~~ then we need to find 2's complement of the final output.

V.3mp



RTL code of Booth's Algorithm.

```
1:  $A \leftarrow 0, Q_{-1} \leftarrow 0, \text{count} \leftarrow n$   
if ( $Q_0, Q_{-1} = 01$ )  
    2:  $A \leftarrow A + M$   
else if ( $Q_0, Q_{-1} = 10$ )  
    2:  $A \leftarrow A - M$   
    3:  $\text{ashr}(A, Q, Q_{-1})$   
    3:  $\text{count} \leftarrow \text{count} - 1$   
if ( $\text{count} > 0$ )  
    4. go to 2  
else  
    4. FINISH
```


111
1111 5) Trace the ~~RTL~~ multiplication of given numbers using RTL code for Booth's algorithm.

i) 4 and -2

ii) -5 and 4

iii) 9 and -3

iv) 4 and -7

v) -7 and -14

vi) (-4) and (-5)

vii) (+14) and (-14) → assume 5 bit register.

} only one will be asked for 7 marks.

6) Trace the multiplication of (-5) and (4) using RTL code for Booth's algorithm.

soln: here,

Multiplicand (M) = -5 → (1011)

Multiplier (Q) = 4 → (0100) ~~11~~

(-M) = (5) → (0101)

here the register size is 4.

Initiate with 0

Date: _____

Page: _____

AL	Q	Q-1	Operation	Cycle
0000	0100	0		
↓ ↓ ↓ ↓	↓ ↓ ↓ ↓	↓	$Q_0 \cdot Q_{-1} = 00$ $Q \leftarrow \text{ashr}(AL, Q, Q_{-1})$	1
0000	0010	0		
↓ ↓ ↓ ↓	↓ ↓ ↓ ↓	↓	$Q_0 \cdot Q_{-1} = 00$ $Q \leftarrow \text{ashr}(AL, Q, Q_{-1})$	2
0000	0001	0		
↓ ↓ ↓ ↓	↓ ↓ ↓ ↓	↓		
0010	1000	1	$Q_0 \cdot Q_{-1} = 10$ $AC \leftarrow AC - M$ $\{AC + (-M)\}$ $Q \leftarrow \text{ashr}(AL, Q, Q_{-1})$	3
0101	0001	0		
↓ ↓ ↓ ↓	↓ ↓ ↓ ↓	↓		
0010	1000	1		
1101	1000	1	$Q_0 \cdot Q_{-1} = 01$ $AC \leftarrow AC + M$ $Q \leftarrow \text{ashr}(AL, Q, Q_{-1})$	4
↓ ↓ ↓ ↓	↓ ↓ ↓ ↓	↓		
1110	1100	0		

We take $AL \cdot Q$ i.e. 11101100

Since the multiplication is $(-ve) \times (+ve) = -ve$
So we need negative number

Find out 2's complement of result

11101100
00010011
+1

00010100 $\rightarrow$ 20

$\therefore$ Hence, -20 is final answer //

$$5 = 0101$$

$$\begin{array}{r} 0101 \\ +1 \\ \hline 0111 \end{array}$$

$$4 = 0100$$

$$\begin{array}{r} 0101 \\ +1 \\ \hline 0110 \end{array}$$

Date: _____

Page: _____

7) Booth algorithm of (-4) and (-5) .

Soln: Here,

Register Size = 4

Multiplicand (M) = $-4 \Rightarrow (1100)$

$(-M) = 4 \Rightarrow 0100$

Multiplier (Q) = $-5 \Rightarrow (1011)$

$(-Q) = 5 \Rightarrow 0101$

AC	Q	Q-1	Operation	Cycle
0000	1011	0	Initial value	
0100 0100 0100	1011 0101 0101	0 1	$Q_0, Q_{-1} = 10$ $AC \leftarrow AC - M$ $ashr(AC, Q, Q-1)$	1
0001 0001 0001	0010 0010 0010	1 1	$Q_0, Q_{-1} = 11$ $ashr(AC, Q, Q-1)$	2
1101 1101 1101	0010 1001 1001	1 0	$Q_0, Q_{-1} = 01$ $AC \leftarrow AC + M$ $ashr(AC, Q, Q-1)$	3
0010 0010 0010	1001 0100 0100	0 1	$Q_0, Q_{-1} = 10$ $AC \leftarrow AC - M$ $ashr(AC, Q, Q-1)$	4

$$00010100 = 20$$

$$\begin{array}{cc} 2^4 & 2^0 \end{array}$$

$\therefore$ Hence, the ~~sub~~ result is 20.

$$-14 = +14 \\ = 01110 \\ \underline{10001} \\ 10010$$

$+14 = 01110$
bit = 4 and we take +1
 $\therefore$ So 5 bit

Q) Multiply (+14) and (-14) using Booth's algorithm. Take register size = 5.

soln: Here,

$$\text{Multiplicand (M)} = 14 = 01110$$

$$(-M) = (-14) = 10010$$

$$\text{Multiplier (Q)} = (-14) = 10010$$

$$(-Q) = 01110$$

Register Size = 5 = Cycle.

Rough

$$\begin{array}{r} 11001 \\ 01110 \\ \hline 00111 \end{array}$$

AC	Q	Q-1	Operation	Cycle
00000	10010	0	Initial value	
00000	01001	0	$Q_0, Q_{-1} = 00$ $\text{ashr}(A, Q, Q_{-1})$	1
10010	01001	0	$Q_0, Q_{-1} = 10$ $AC \leftarrow AC - M$	2
11001	00100	1	$\text{ashr}(A, Q, Q_{-1})$	
00111	00100	1	$Q_0, Q_{-1} = 01$ $AC \leftarrow AC + M$	3
00011	10010	0	$\text{ashr}(A, Q, Q_{-1})$	
00001	11001	0	$\text{ashr}(A, Q, Q_{-1})$	4
10011	11001	0	$Q_0, Q_{-1} = 10$ $AC \leftarrow AC - M$	5
11001	11100	1	$\text{ashr}(A, Q, Q_{-1})$	

$$11001 \ 11100$$

$$00110 \ 00011$$

+1

$$00110 \ 00100$$

Its equivalent is 196

and we add (-) sign

$$\therefore -196$$

Ans //

III 9) BCD addition or BCD number

Ans: BCD stands for Binary Coded Decimal.

- It is a way of representing decimal numbers using binary digits.
- In BCD, each decimal digit is converted into its equivalent 4-bit binary representation.
- For values 0-9, their representations are the same as in binary.
- However, 4-bit values above 9 are not used in BCD.

Representing 8 in BCD. i.e. 1000

Representing 10 in BCD

1001 → this is wrong.

We can represent it as

0001 0000

1 0

= 10

• BCD addition

- BCD addition is the process of adding two BCD numbers.
- The steps of BCD addition are:-

Step 1: Convert each decimal digit of the number into its 4-bit binary form

Step 2: Make sure both BCD numbers are in 4 bits. Add leading zeros if necessary.

Step 3: Add the digits column by column, just like regular addition.

However, if the sum of the digits exceeds 9, it is necessary to add 6 to the result to convert it back to valid BCD representation.

Example: $8 + 8$

$$\begin{array}{r} 1000 \\ + 1000 \\ \hline 10000 \end{array}$$

In BCD it will be 0001 0000

1 6
i.e. 16 which is wrong.

So add 6 to the sum i.e. 0110

$$\begin{array}{r} 10000 \\ + 0110 \\ \hline 00010110 \\ 1 6 \end{array}$$

i.e. 16

∴ Hence, we perform BCD addition in this way.

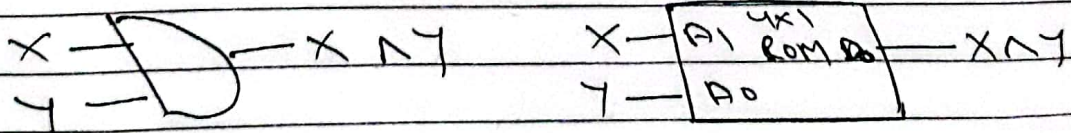
1111

10) What is lookup ROM? Illustrate with an example how it works. (lookup Table)

Ans: Any combinationed circuit that can be implemented by ROM is lookup ROM or lookup Table.

- Simply it is a component that stores data in a fixed way.
- It is used to provide predefined outputs based on given inputs, similar to the table that holds specific answers for certain questions.
- We provide an address to the ROM (like a location in the table) and the ROM gives us the data that's stored there.
- They are used because they provide answers very quickly.
- Inputs to the combinationed circuit serve as the address input of the ROM.
- Data outputs of the ROM corresponds to the output of the combinationed circuit.

Example: Consider a 4x1 ROM programmed to mimic a two input AND gate.



(as) AND gate

Address $x \ y$		Data z
0	0	0
0	1	0
1	0	0
1	1	1

b) lookup ROM equivalent

Here, the lookup ROM stores the output for all input combinations.

When we provide an address (input combination), the ROM gives us the corresponding output.

It instantly provides the AND gate output for any input combination.

- 11) Describe in brief demonstrating a lookup ROM working as XOR gate.
soln:-

The XOR gate is given as

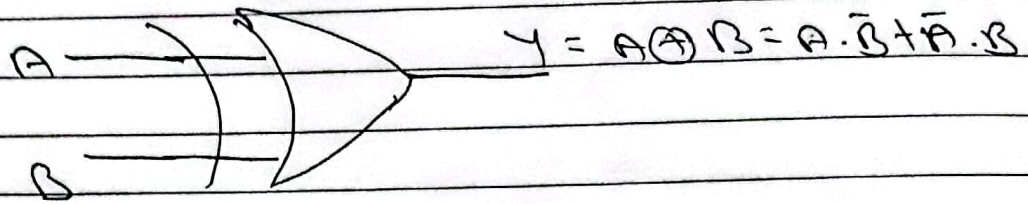
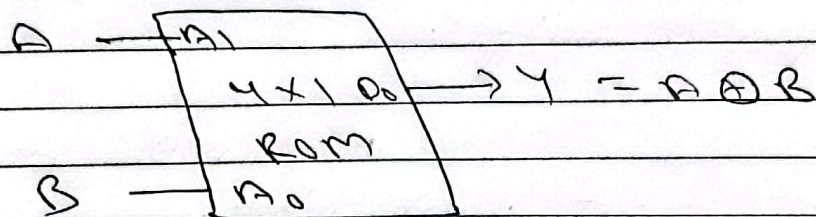


Fig: XOR gate

Now



Address		Data
A_1	A_0	Y
0	0	0
0	1	1
1	0	1
1	1	0

Fig: lookup equivalent

{Explanation is Que no 10}

12) Show the memory content of a lookup ROM equivalent to two input OR gate.

{ Ans $\rightarrow$ Same like 11 but for OR gate }

13) Design a lookup ROM to implement the function $f = xy + y'z$.

$\rightarrow$ Here the function has x, y, z as variables.
So the combinational circuit will be,

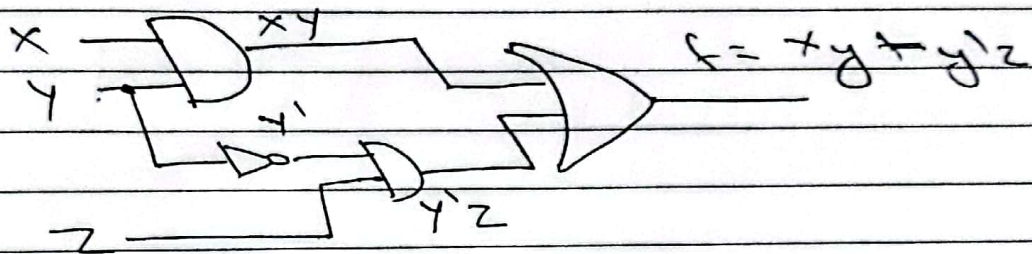
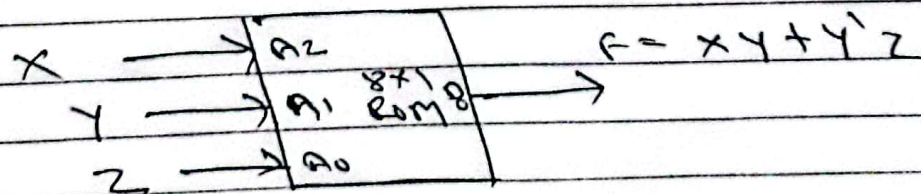


Fig: Combinational Circuit

We have three inputs and one output
that means $n = 3$ input & output $m = 1$

Size of ROM will be $2^n \times m = 2^3 \times 1$

i.e $2^3 \times 1 = 8 \times 1$ ROM



P.T.O $\rightarrow$

Now, the lookup table can be constructed,

Address			Data
A_0	A_1	A_2	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

14) Show the lookup ROM implementation for the function $f = x_1y_1y_2z_2$.

→ Same like 13.

1111

15) What is Wallace Tree?

or,

Wallace Tree Multiplier.

or

Describe in brief demonstrating a 3×3 multiplier being constructed using Wallace tree and carry save adder.

Ans: The Wallace Tree is a method ~~of~~ for optimizing and accelerating the multiplication of large binary numbers in digital circuits.

- It helps to break the multiplication process into smaller, easier steps.
- Here, big number is ~~not~~ broken down into small numbers as it is easier to multiply small numbers.
- Wallace Tree adds up the products of these smaller numbers, creating partial sums that are combined to give the final answer.
- This technique is super fast and helps computers do large multiplications quickly.
- It is called a 'tree' because the partial sums form a structure that looks like branches coming together forming the final solution.
- It maximizes parallelism, allowing multiple calculations to happen at once in parallel, speeding up the process.
- It is used in computer parts and devices where fast multiplication is crucial.

Example: 3x3 multiplier being created using Wallace Tree and carry save adder.

Soln: Here,

$$\text{Let } x = 111 \\ y = 110$$

$$(x) \times (y) ; \quad \begin{array}{r} 111 \\ \times 110 \\ \hline \end{array}$$

Partial Product

Add zero in $\boxed{00}000 \leftarrow PP0$

Blank to make $\boxed{0}1110 \leftarrow PP1$

all equal bit $+ 11100 \leftarrow PP2$

Sum(s) 10010

Carry(c) 01100

We add carry to sum by shifting carry to left

10010

$+ 011000$

$101010 \rightarrow \text{Final Product}$

Figure:

PP0
(00000)

PP1
(01110)

5-bit carry save adder

$\leftarrow PP2 (11100)$

c(011000)

s(10010)

6-bit

cin $\leftarrow 0$

Parallel adder

Final product
(101010)

16) What is numeric format?

Ans: A numeric format is a way of representing numbers in a specific structure so that computer can understand and work with them accurately.

→ Numeric formats ensure that numbers are stored, processed, and displayed correctly in different computer systems and applications.

→ Different types of numeric formats include whole numbers, decimals, fractions, etc.

17) Explain floating point numbers.

Ans: Floating points are a way to represent real numbers with decimal points in a computer.

→ It is like a ~~scientific~~ similar to a scientific notation.

→ Floating point number can be represented in different ways i.e. -123.4567

$$\text{or, } = -1.234567 \times 10^2$$

$$\text{or, } = -123456.7 \times 10^{-3}$$

→ But computers can be efficient if each number can have only one representation (unique).

→ So, floating point numbers must be normalized i.e. each number significant is a fraction with (mantissa) no leading zeros.

→ Thus, the only valid floating point representation for this number is $- .1234567 \times 10^3$

118) Explain IEEE 754 floating point standard.

Ans. IEEE 754 standard is a set of rules that helps computers work with decimal numbers efficiently.

→ This standard is used in virtually all CPUs that have floating point capability.

→ IEEE 754 splits a real number into two parts: the mantissa (actual digit) and exponent ~~the~~.

Example: (2.54×10^3)

Mantissa = 2.54

Exponent = 3

→ IEEE 754 standard specifies two precision ~~&~~ for floating point ~~capability~~ numbers.

1. single precision number having 32 bits.

2. double precision number having 64 bits.

In single precision,

1 bit for sign, 8 bit for exponent and 23 bit for significand (mantissa).

In double precision,

1 bit for sign, 11 bit for exponent and 52 bit for significand (mantissa).

→ Real-world calculations often involve real numbers: measurements, money, scientific data.

→ IEEE 754 helps computers manage these calculations with precision.

1.1) What is Arithmetic Pipelining? Describe in brief with an example.

Ans: Arithmetic Pipelining is a technique used to speed up the execution of arithmetic operations by breaking them into smaller stages and overlapping these stages to achieve better efficiency.

→ Here, different tasks are performed in parallel to maximize throughput.

→ In arithmetic pipelining, a complex calculation is divided into stages, each handling a specific part of the operation.

→ These stages are often executed simultaneously.

→ As one stage completes its work, the next stage starts with a new set of data.

→ This overlap reduces idle time, making calculations faster and more efficient.

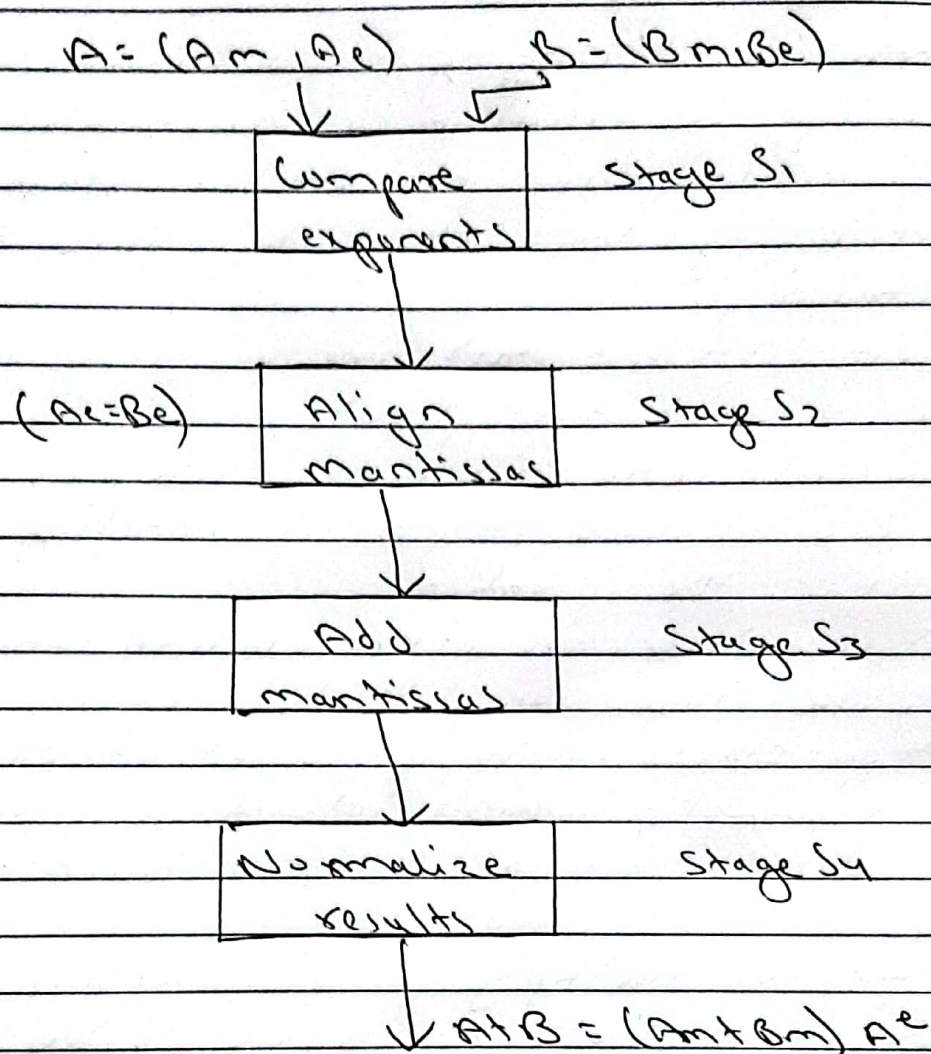
→ When we multiply large numbers.

↳ In non-pipelined approach, we do the entire multiplication step by step.

↳ In pipelining we divide the process, one stage for multiplication, another for addition and so on.

→ Pipelining significantly speeds up arithmetic operations as the different stages work in parallel, reducing the overall time needed to complete a series of calculations.

Example: Floating point addition using arithmetic pipeline.



~~The following sub operations~~

Fig: Pipeline for floating point addition of two numbers.

- The following sub operations are performed in this case:
- i) Compare the exponents
 - ii) Align the mantissas
 - iii) Add or subtract the mantissas
 - iv) Normalize the result

let us consider two numbers

$$X = 0.321 \times 10^3 \quad Y = 0.450 \times 10^2$$

First the two exponents is compared and aligned.
We need to make exponent of Y also 10^3 .

$$Y = 0.045 \times 10^3$$

The two numbers are added to produce

$$\text{Result} = 0.366 \times 10^3$$

ans //